

Course 2: Supervised Learning



IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

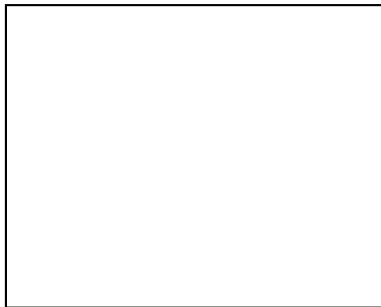
Last session

- 1 AI definition
- 2 Applications & Open Issues
- 3 Deep learning
- 4 Foundation models

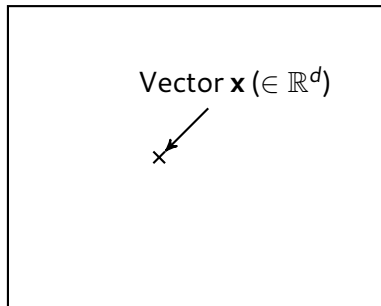
Today's session

- Learning from labeled examples
- Challenges of supervised learning
- Evaluation: ground truth and metrics

Vector space ($\mathbb{R}^d$)



Vector space ($\mathbb{R}^d$)

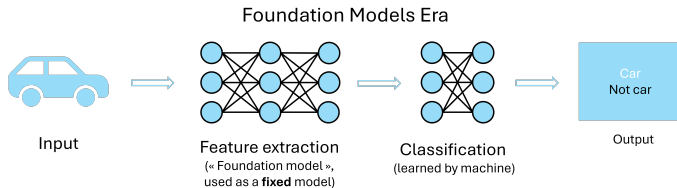
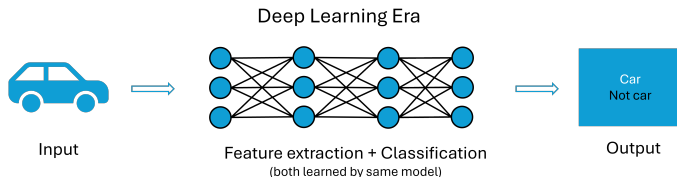
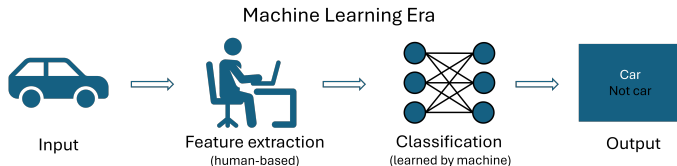


Notations

Vector space ($\mathbb{R}^d$)



What is the vector x ? (1/2)



What is the vector x ? (2/2)

Traditional Machine Learning

→ x is generally a feature representation of the data

Ex: edges in the image, spectrograms in audio, etc.

Deep Learning

→ x is the data, or a small transformation of the data

The era of Foundation models

→ x is the projection of data in an **embedding space**

- Advantage: richer semantically than the original image
- Disadvantage: Not interpretable nor easily understandable

Vector space ($\mathbb{R}^d$)

Vector $\mathbf{x} \in \mathbb{R}^d$



What is the vector x ? (2/2)

Traditional Machine Learning

→ x is generally a feature representation of the data

Ex: edges in the image, spectrograms in audio, etc.

Deep Learning

→ x is the data, or a small transformation of the data

The era of Foundation models

→ x is the projection of data in an **embedding space**

- Advantage: richer semantically than the original image
- Disadvantage: Not interpretable nor easily understandable

Vector space ($\mathbb{R}^d$)

Vector $\mathbf{x} \in \mathbb{R}^d$



What is the vector $\mathbf{x}$? (2/2)

Traditional Machine Learning

→ $\mathbf{x}$ is generally a feature representation of the data

Ex: edges in the image, spectrograms in audio, etc.

Deep Learning

→ $\mathbf{x}$ is the data, or a small transformation of the data

The era of Foundation models

→ $\mathbf{x}$ is the projection of data in an **embedding space**

- Advantage: richer semantically than the original image
- Disadvantage: Not interpretable nor easily understandable

Vector space ($\mathbb{R}^d$)

Vector $\mathbf{x} \in \mathbb{R}^d$



- 1 Learning from labeled examples
- 2 Challenges of supervised learning
- 3 Evaluation: ground truth and metrics

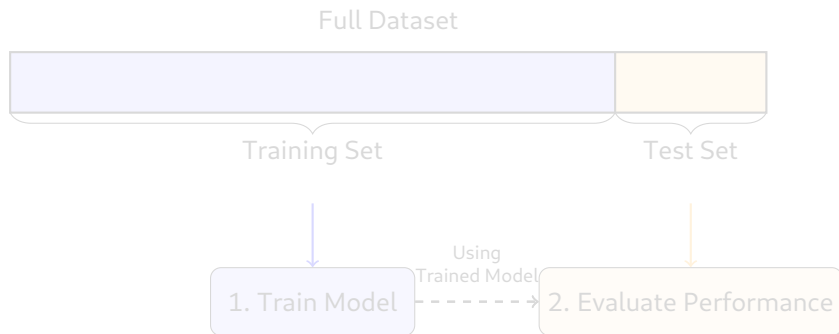
Learning: Training and Test sets

The Training Set

Data used by the model to **learn** from examples.

The Test Set

A smaller, hold-out portion that the model has **never seen**. Used to **evaluate** its performance.



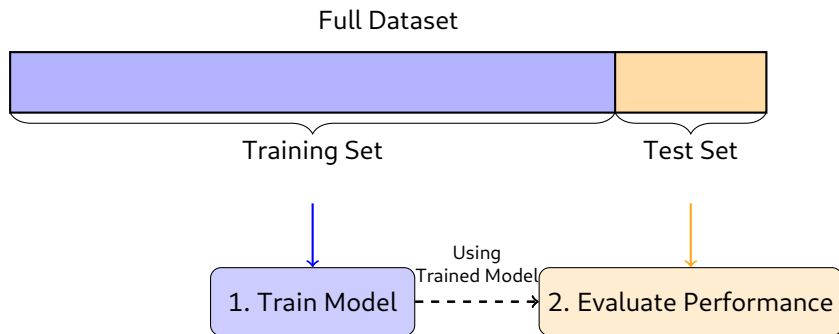
Learning: Training and Test sets

The Training Set

Data used by the model to **learn** from examples.

The Test Set

A smaller, hold-out portion that the model has **never seen**. Used to **evaluate** its performance.



Definition

Given a training set, composed of:

- **x**: inputs (raw signals or feature vectors (e.g. embeddings))
- **y: labels** (annotated by humans)

Learn:

- a function $f()$ such that $\mathbf{y} \approx f(\mathbf{x})$ (denoted as $\hat{\mathbf{y}}$)
 $\Rightarrow f()$ is **learned** by the Machine Learning algorithm
- Ideally, $f()$ should **generalize** ($\neq$ memorize) to unlabeled examples.

Supervised learning

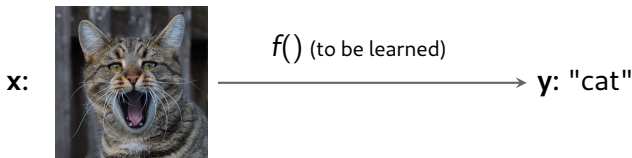
Definition

Given a training set, composed of:

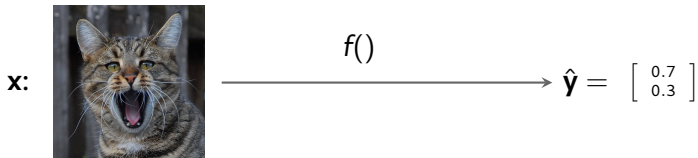
- **x**: inputs (raw signals or feature vectors (e.g. embeddings))
- **y: labels** (annotated by humans)

Learn:

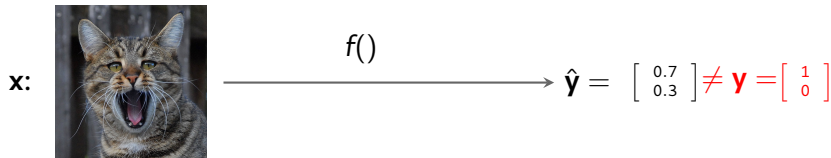
- a function $f()$ such that $\mathbf{y} \approx f(\mathbf{x})$ (denoted as $\hat{\mathbf{y}}$)
 $\Rightarrow f()$ is **learned** by the Machine Learning algorithm
- Ideally, $f()$ should **generalize** ($\neq$ memorize) to unlabeled examples.



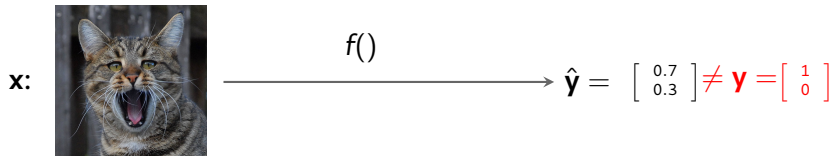
Supervised learning: in practice



Supervised learning: in practice



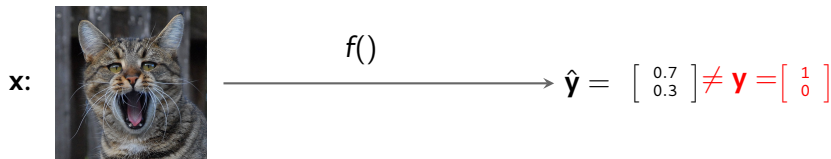
Supervised learning: in practice



Loss

- Here, labels are encoded as one-hot vectors,
- We compute a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$
- Training consist in minimizing the loss!

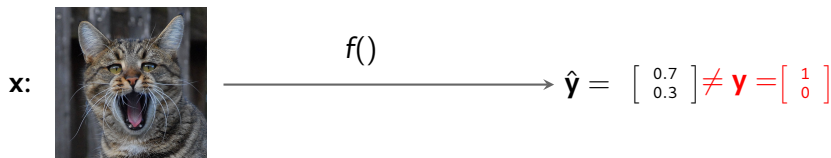
Supervised learning: in practice



Loss

- Here, labels are encoded as one-hot vectors,
- We compute a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$
 - Euclidean distance: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \sum_{i=1}^D (\hat{\mathbf{y}}_i - \mathbf{y}_i)^2$
 - Cross-entropy: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{i=1}^D \mathbf{y}_i \log(\hat{\mathbf{y}}_i)$
- Training consist in minimizing the loss!

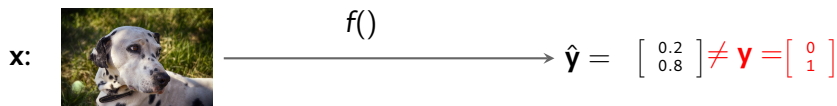
Supervised learning: in practice



Loss

- Here, labels are encoded as one-hot vectors,
- We compute a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$
 - Euclidean distance: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \sum_{i=1}^D (\hat{\mathbf{y}}_i - \mathbf{y}_i)^2$
 - Cross-entropy: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{i=1}^D \mathbf{y}_i \log(\hat{\mathbf{y}}_i)$
- Training consist in minimizing the loss!
 $\Rightarrow$ Here, one can use gradient descent (see class 1.)

Supervised learning: in practice

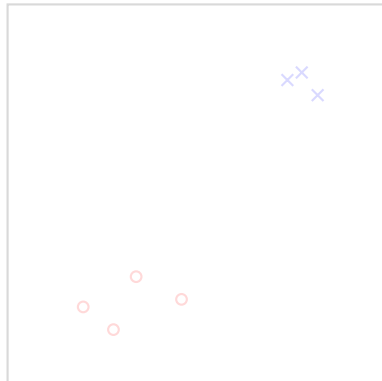


Loss

- Here, labels are encoded as one-hot vectors,
- We compute a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$
 - Euclidean distance: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \sum_{i=1}^D (\hat{\mathbf{y}}_i - \mathbf{y}_i)^2$
 - Cross-entropy: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{i=1}^D \mathbf{y}_i \log(\hat{\mathbf{y}}_i)$
- Training consist in minimizing the loss!
 $\Rightarrow$ Here, one can use gradient descent (see class 1.)

Examples

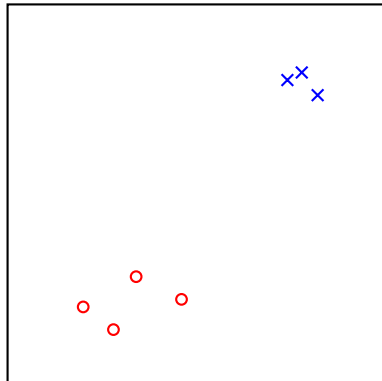
- **Classification** (y is categorical)
- **Regression** (y is scalar)
- Tons of applications:
 - Pattern recognition,
 - Prediction...



Supervised learning

Examples

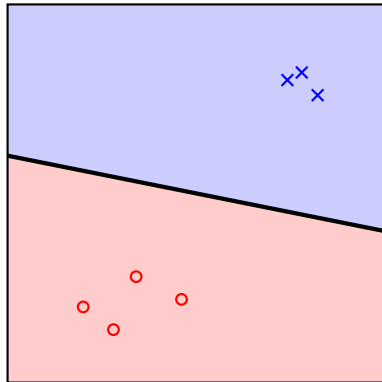
- **Classification** (y is categorical)
- **Regression** (y is scalar)
- Tons of applications:
 - Pattern recognition,
 - Prediction...



Supervised learning

Examples

- **Classification** (y is categorical)
- **Regression** (y is scalar)
- Tons of applications:
 - Pattern recognition,
 - Prediction...

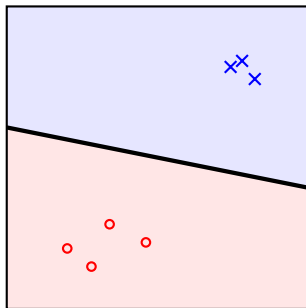


- 1 Learning from labeled examples
- 2 Challenges of supervised learning
- 3 Evaluation: ground truth and metrics

Infinite solutions and priors

An ill-defined problem

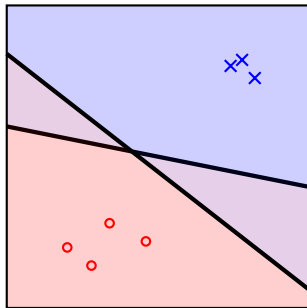
- An infinity of potential solutions, one must be the “best one” but is unreachable,
- $\Rightarrow$ requires **priors or constraints**.
- Training on more data is a way to reduce the number of potential solutions, but it is not always available.



Infinite solutions and priors

An ill-defined problem

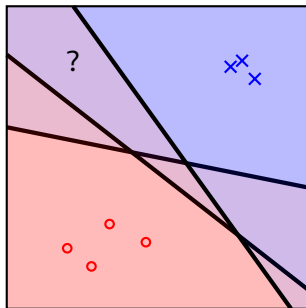
- An infinity of potential solutions, one must be the “best one” but is unreachable,
- $\Rightarrow$ requires **priors or constraints**.
- Training on more data is a way to reduce the number of potential solutions, but it is not always available.



Infinite solutions and priors

An ill-defined problem

- An infinity of potential solutions, one must be the “best one” but is unreachable,
- $\Rightarrow$ requires **priors or constraints**.
- Training on more data is a way to reduce the number of potential solutions, but it is not always available.

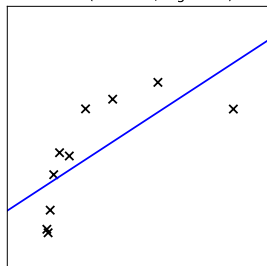


Bias/variance trade-off

A **simple** solution that almost matches is better than a complex one that fully matches!

- Bias: Error from **erroneous assumptions** in the learning algorithm.
- Variance: Error from **sensitivity to small fluctuations** in the training set.

Polynomial regression,
 $d = 1$ (under-fit; high bias)

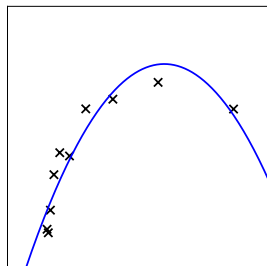


Bias/variance trade-off

A **simple** solution that almost matches is better than a complex one that fully matches!

- Bias: Error from **erroneous assumptions** in the learning algorithm.
- Variance: Error from **sensitivity to small fluctuations** in the training set.

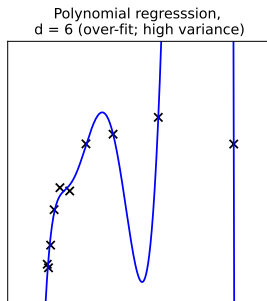
Polynomial regression,
 $d = 2$



Bias/variance trade-off

A **simple** solution that almost matches is better than a complex one that fully matches!

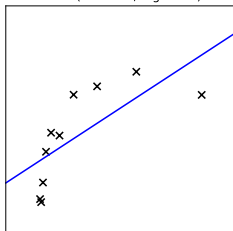
- Bias: Error from **erroneous assumptions** in the learning algorithm.
- Variance: Error from **sensitivity to small fluctuations** in the training set.



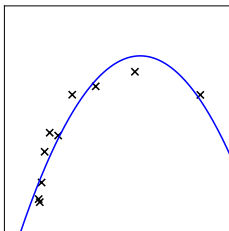
Bias/variance trade-off

A **simple** solution that almost matches is better than a complex one that fully matches!

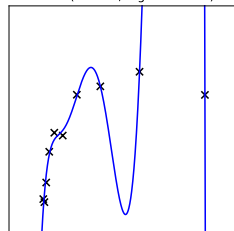
Polynomial regression,
 $d = 1$ (under-fit; high bias)



Polynomial regression,
 $d = 2$



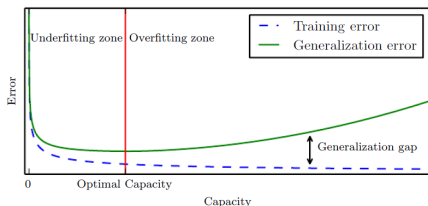
Polynomial regression,
 $d = 6$ (over-fit; high variance)



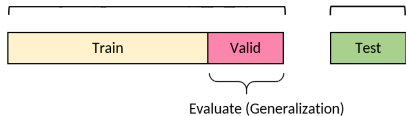
Overfitting

Overfitting

Mimicking is not learning: **overfitting** problem.



X n_epochs
Iterate on epochs
To tune hyperparameters



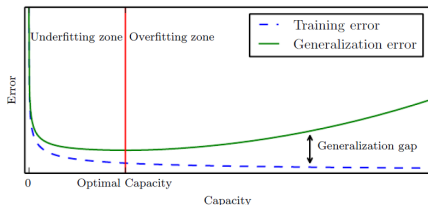
To detect overfitting, you may use a validation set:

- Split the **training** dataset into two parts:
- The first part is used to train,
- The second part is used to validate (Validation Set), i.e. check for overfitting.

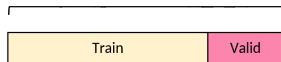
Overfitting

Overfitting

Mimicking is not learning: **overfitting** problem.



X n_epochs
Iterate on epochs
To tune hyperparameters



Once to test
performances



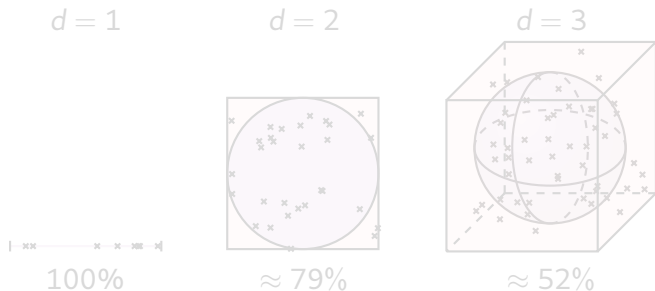
To detect overfitting, you may use a validation set:

- Split the **training** dataset into two parts:
- The first part is used to train,
- The second part is used to validate (Validation Set), i.e. check for overfitting.

High-dimensional data (1/2)

Curse of dimensionality

- Geometry is not intuitive in **high dimension**,
- Efficient methods in 2D are not necessarily still valid.



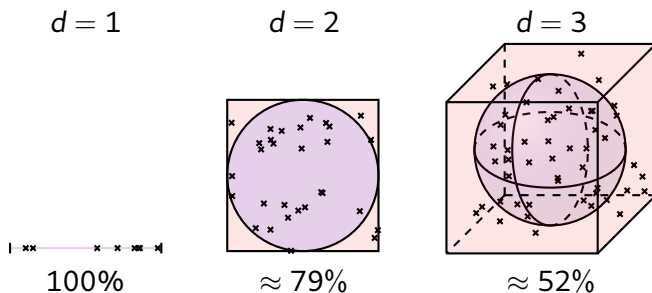
$$V_d^s = \frac{\pi^{d/2} R^d}{\Gamma(d/2 + 1)} \text{ versus } V_d^c = (2R)^d$$

see <https://youtu.be/dZrGXYty3qc?t=533>

High-dimensional data (1/2)

Curse of dimensionality

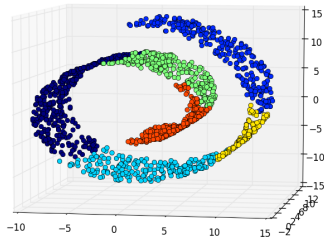
- Geometry is not intuitive in **high dimension**,
- Efficient methods in 2D are not necessarily still valid.



$$V_d^s = \frac{\pi^{d/2} R^d}{\Gamma(d/2 + 1)} \text{ versus } V_d^c = (2R)^d$$

see <https://youtu.be/dZrGXYty3qc?t=533>

Riemannian manifolds



The natural space of data may not always be suited to represent data!
⇒ This is why we generally use feature descriptions, or embeddings, to represent data!

Time and material resources

Computation time

Example on ImageNet, simply going through all images:

- $n = 10.000.000$, $d \approx 1.000.000$,
- $\approx 10^{13}$ elementary operations,
- $\approx 2\text{h}45$ on a modern processor.

Scalability

- Finding the best solution to a problem would be feasible with unlimited computation time,
- But searching through the space of possible functions is often **untractable**,
- Solutions must be computationally reasonable, which is the true challenge today.

Time and material resources

Computation time

Example on ImageNet, simply going through all images:

- $n = 10.000.000$, $d \approx 1.000.000$,
- $\approx 10^{13}$ elementary operations,
- $\approx 2\text{h}45$ on a modern processor.

Scalability

- Finding the best solution to a problem would be feasible with unlimited computation time,
- But searching through the space of possible functions is often **untractable**,
- Solutions must be computationally reasonable, which is the true challenge today.

- 1 Learning from labeled examples
- 2 Challenges of supervised learning
- 3 Evaluation: ground truth and metrics

Evaluation: ground truth vs. estimation

Ground truth y

The **reference** answer: given by a human expert, a sensor, or a measurement protocol.

⇒ what we *assume* to be correct.

Estimation $\hat{y} = f(x)$

The answer **produced by the model**.

⇒ available for any x , even without label.

Test example	x_1	x_2	x_3	x_4	x_5
Ground truth y	cat	dog	cat	dog	cat
Estimation $\hat{y}$	cat	dog	dog	dog	cat
	✓	✓	✗	✓	✓

Why do we need metrics?

The problem

Comparing $\hat{\mathbf{y}}$ and $\mathbf{y}$ example by example does not scale: a test set has hundreds or thousands of examples.

⇒ we need to **summarise** the comparison into a few numbers.

What we use these numbers for

- **Choosing** between two algorithms (KNN or a decision tree?),
- **Tuning** hyperparameters (which value of k ?),
- **Detecting** overfitting (compare train score vs. test score),
- **Deciding** whether the model is good enough to be used at all.

Loss $\neq$ metric

Loss $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$

What the algorithm **minimises** during training. Must be optimisable, not necessarily interpretable.

Metric

What **you** report and interpret, on the test set. Must be meaningful for the application.

Why do we need metrics?

The problem

Comparing $\hat{\mathbf{y}}$ and $\mathbf{y}$ example by example does not scale: a test set has hundreds or thousands of examples.

⇒ we need to **summarise** the comparison into a few numbers.

What we use these numbers for

- **Choosing** between two algorithms (KNN or a decision tree?),
- **Tuning** hyperparameters (which value of k ?),
- **Detecting** overfitting (compare train score vs. test score),
- **Deciding** whether the model is good enough to be used at all.

Loss $\neq$ metric

Loss $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$

What the algorithm **minimises** during training. Must be optimisable, not necessarily interpretable.

Metric

What **you** report and interpret, on the test set. Must be meaningful for the application.

Why do we need metrics?

The problem

Comparing $\hat{\mathbf{y}}$ and $\mathbf{y}$ example by example does not scale: a test set has hundreds or thousands of examples.

⇒ we need to **summarise** the comparison into a few numbers.

What we use these numbers for

- **Choosing** between two algorithms (KNN or a decision tree?),
- **Tuning** hyperparameters (which value of k ?),
- **Detecting** overfitting (compare train score vs. test score),
- **Deciding** whether the model is good enough to be used at all.

Loss $\neq$ metric

Loss $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$

What the algorithm **minimises** during training. Must be optimisable, not necessarily interpretable.

Metric

What **you** report and interpret, on the test set. Must be meaningful for the application.

A first metric: accuracy

Definition

The fraction of test examples that are correctly predicted:

$$\text{accuracy} = \frac{\text{number of correct predictions}}{\text{number of examples}} = \frac{1}{n} \sum_{i=1}^n \mathbf{1}[\hat{y}_i = y_i]$$

Test example	x_1	x_2	x_3	x_4	x_5
Ground truth y	cat	dog	cat	dog	cat
Estimation $\hat{y}$	cat	dog	dog	dog	cat
	✓	✓	✗	✓	✓

$$\text{accuracy} = 4/5 = 80\%$$

A first metric: accuracy

Definition

The fraction of test examples that are correctly predicted:

$$\text{accuracy} = \frac{\text{number of correct predictions}}{\text{number of examples}} = \frac{1}{n} \sum_{i=1}^n \mathbf{1}[\hat{y}_i = y_i]$$

Test example	x_1	x_2	x_3	x_4	x_5
Ground truth y	cat	dog	cat	dog	cat
Estimation $\hat{y}$	cat	dog	dog	dog	cat
	✓	✓	✗	✓	✓

$$\text{accuracy} = 4/5 = 80\%$$

A first metric: accuracy

Definition

The fraction of test examples that are correctly predicted:

$$\text{accuracy} = \frac{\text{number of correct predictions}}{\text{number of examples}} = \frac{1}{n} \sum_{i=1}^n \mathbf{1}[\hat{y}_i = y_i]$$

Test example	x_1	x_2	x_3	x_4	x_5
Ground truth y	cat	dog	cat	dog	cat
Estimation $\hat{y}$	cat	dog	dog	dog	cat
	✓	✓	✗	✓	✓

$$\text{accuracy} = 4/5 = \mathbf{80\%}$$

Why accuracy is not enough

A screening test

1 000 patients, 10 of them are actually sick.



990 healthy

10 sick

A very lazy model: "everybody is healthy"

- Accuracy = $990/1000 = 99\%$,
- Number of sick patients detected: 0.

⇒ we need metrics that look at **each class separately**,
and at the **type** of error being made.

Why accuracy is not enough

A screening test

1 000 patients, 10 of them are actually sick.



990 healthy

10 sick

A very lazy model: "everybody is healthy"

- Accuracy = $990/1000 = 99\%$,
- Number of sick patients detected: **0**.

⇒ we need metrics that look at **each class separately**,
and at the **type** of error being made.

Why accuracy is not enough

A screening test

1 000 patients, 10 of them are actually sick.



990 healthy

10 sick

A very lazy model: "everybody is healthy"

- Accuracy = $990/1000 = 99\%$,
- Number of sick patients detected: **0**.

⇒ we need metrics that look at **each class separately**,
and at the **type** of error being made.

Precision and recall

Definition (for **one** class, chosen as “positive”)

$$\text{precision} = \frac{TP}{TP + FP}$$

Among the examples **predicted** positive, how many really are?

$$\text{recall} = \frac{TP}{TP + FN}$$

Among the **truly** positive examples, how many are found?

TP: true positives, *FP*: false positives (*false alarms*), *FN*: false negatives (*missed*).

Test example	x_1	x_2	x_3	x_4	x_5
Ground truth y	cat	dog	cat	dog	cat
Estimation $\hat{y}$	cat	dog	dog	dog	cat

With “cat” as positive: $TP = 2$, $FP = 0$, $FN = 1$
precision = $2/2 = 100\%$ but recall = $2/3 \approx 67\%$

F1-score: summarising both in one number

$$F_1 = 2 \cdot \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \quad \text{here } F_1 = 2 \cdot \frac{1 \times 0.67}{1 + 0.67} \approx 80\%$$

Harmonic mean $\Rightarrow$ collapses as soon as **one** of the two is low.

Precision and recall

Definition (for **one** class, chosen as “positive”)

$$\text{precision} = \frac{TP}{TP + FP}$$

Among the examples **predicted** positive, how many really are?

$$\text{recall} = \frac{TP}{TP + FN}$$

Among the **truly** positive examples, how many are found?

TP: true positives, *FP*: false positives (*false alarms*), *FN*: false negatives (*missed*).

Test example	x_1	x_2	x_3	x_4	x_5
Ground truth y	cat	dog	cat	dog	cat
Estimation $\hat{y}$	cat	dog	dog	dog	cat

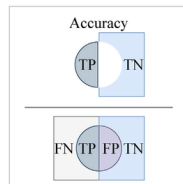
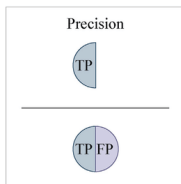
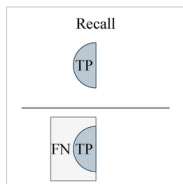
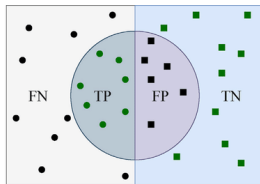
With “cat” as positive: $TP = 2$, $FP = 0$, $FN = 1$
precision = $2/2 = \mathbf{100\%}$ but recall = $2/3 \approx \mathbf{67\%}$

F1-score: summarising both in one number

$$F_1 = 2 \cdot \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \quad \text{here } F_1 = 2 \cdot \frac{1 \times 0.67}{1 + 0.67} \approx \mathbf{80\%}$$

Harmonic mean $\Rightarrow$ collapses as soon as **one** of the two is low.

Precision and recall: the intuition



https://www.researchgate.net/publication/346129022_Overview_of_Machine_Learning_Part_1/figures

What about several classes?

One class at a time ("one-vs-rest")

For each class c : treat c as **positive** and **all the others** as negative.
⇒ one precision and one recall **per class**.

	precision	recall
class "cat"	$2/2 = 100\%$	$2/3 \approx 67\%$
class "dog"	$2/3 \approx 67\%$	$2/2 = 100\%$

Averaging into a single number

- **macro**: plain mean over classes → every class counts the same,
- **weighted**: mean weighted by the number of examples per class,
- **micro**: pool the TP , FP , FN of all classes.

Here: macro precision $\approx 83\%$, micro precision = 80%.

What about several classes?

One class at a time ("one-vs-rest")

For each class c : treat c as **positive** and **all the others** as negative.
⇒ one precision and one recall **per class**.

	precision	recall
class "cat"	$2/2 = 100\%$	$2/3 \approx 67\%$
class "dog"	$2/3 \approx 67\%$	$2/2 = 100\%$

Averaging into a single number

- **macro**: plain mean over classes → every class counts the same,
- **weighted**: mean weighted by the number of examples per class,
- **micro**: pool the TP , FP , FN of all classes.

Here: macro precision $\approx 83\%$, micro precision = 80%.

What about several classes?

One class at a time ("one-vs-rest")

For each class c : treat c as **positive** and **all the others** as negative.
⇒ one precision and one recall **per class**.

	precision	recall
class "cat"	$2/2 = 100\%$	$2/3 \approx 67\%$
class "dog"	$2/3 \approx 67\%$	$2/2 = 100\%$

Averaging into a single number

- **macro**: plain mean over classes → every class counts the same,
- **weighted**: mean weighted by the number of examples per class,
- **micro**: pool the TP , FP , FN of all classes.

Here: macro precision $\approx 83\%$, micro precision = 80%.

Confusion matrix

Confusion matrix

An easy way to visualise all the errors at once is the confusion matrix:

		Ground truth		
		+	-	
Predicted	+	True positive (TP)	False positive (FP)	Precision = $TP / (TP + FP)$
	-	False negative (FN)	True negative (TN)	
		Recall = $TP / (TP + FN)$		Accuracy = $(TP + TN) / (TP + FP + TN + FN)$

https://www.researchgate.net/publication/334840641_A_cloud_detection_algorithm_for_satellite_imagery_based_on_deep_learning/figures?lo=1

Lab Session 2: Supervised Learning

Part 1 — Toy dataset, with `sklearn`

- Generate data with `make_blobs`, split it with `train_test_split`,
- The `sklearn` API: `fit` / `predict` / `score`,
- A first classifier (K-Nearest Neighbors): influence of k on train and test scores $\Rightarrow$ bias/variance in practice,
- Metrics: `classification_report` and `confusion_matrix`,
- Visualise the **decision boundaries**.

Then, do the same for a different dataset and a different classifier.
Compare all results, and notice that they significantly differ!

Part 2 — Your own project data

Apply the very same pipeline to the dataset of **your project**.

$\Rightarrow$ define your task as a classification (or regression) problem, check your class balance, and choose the metrics that make sense for your application.

Lab Session 2: Supervised Learning

Part 1 — Toy dataset, with sklearn

- Generate data with `make_blobs`, split it with `train_test_split`,
- The sklearn API: `fit` / `predict` / `score`,
- A first classifier (K-Nearest Neighbors): influence of k on train and test scores $\Rightarrow$ bias/variance in practice,
- Metrics: `classification_report` and `confusion_matrix`,
- Visualise the **decision boundaries**.

Then, do the same for a different dataset and a different classifier.
Compare all results, and notice that they significantly differ!

Part 2 — Your own project data

Apply the very same pipeline to the dataset of **your project**.

$\Rightarrow$ define your task as a classification (or regression) problem, check your class balance, and choose the metrics that make sense for your application.